

LAPORAN TUGAS
BAHASA PEMROGRAMAN 3
(Dosen : Dede Husen, M.Kom.)

Modul 10
Sensor



Nama : Muhammad Rizal Nurfirdaus
NIM : 20230810088
Kelas : TINFC-2023-04

TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN

PRE-TEST

1. Sebutkan 4 contoh sensor yang ada pada smartphone ?

Empat contoh sensor yang umum terdapat pada smartphone adalah:

- a. Sensor Accelerometer
- b. Sensor Gyroscope
- c. Sensor Proximity
- d. Sensor Light (Ambient Light Sensor)

2. Jelaskan fungsi dari 4 sensor yang ada pada smartphone Anda!

- **Sensor Accelerometer** berfungsi untuk mendeteksi pergerakan dan perubahan posisi smartphone, seperti saat layar berpindah dari mode potret ke lanskap.
- **Sensor Gyroscope** berfungsi untuk mendeteksi rotasi atau perputaran perangkat secara lebih akurat, biasanya digunakan pada game, navigasi, dan aplikasi virtual reality.
- **Sensor Proximity** berfungsi untuk mendeteksi keberadaan objek di dekat layar, misalnya saat menelepon, sensor ini akan mematikan layar agar tidak tersentuh secara tidak sengaja.
- **Sensor Light (Ambient Light Sensor)** berfungsi untuk mendeteksi intensitas cahaya di sekitar smartphone guna menyesuaikan tingkat kecerahan layar.

3. Sensor apakah yang berfungsi untuk mengatur kecerahan layar secara otomatis?

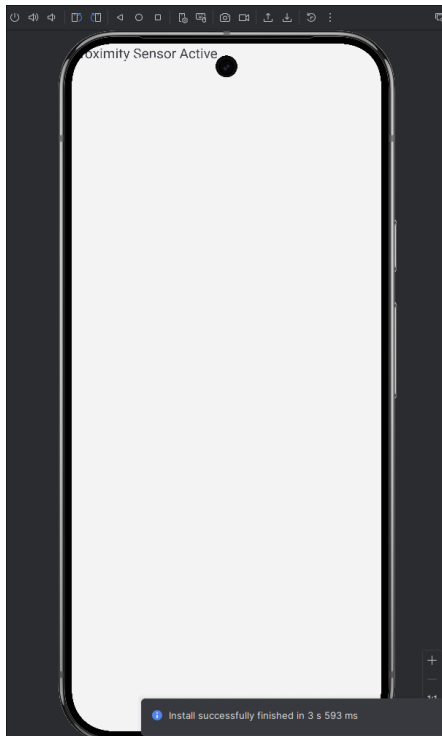
Sensor yang berfungsi untuk mengatur kecerahan layar secara otomatis adalah **sensor cahaya (Ambient Light Sensor)**.

Sensor ini bekerja dengan mendeteksi intensitas cahaya di sekitar perangkat. Jika kondisi lingkungan terang, layar akan otomatis menjadi lebih cerah, sedangkan saat berada di tempat gelap, kecerahan layar akan dikurangi agar lebih nyaman di mata dan menghemat daya baterai.

PRAKTIKUM

Source Code :

https://github.com/MuhammadRizalNurfirdaus/Praktikum_BP3/tree/a46c56355976aa7151ea1820660716dedb652110/Modul10_Sensor



Analisis :

Gambaran Umum Proyek

Proyek ini merupakan aplikasi Android yang dibangun menggunakan sistem build Gradle, dengan basis bahasa pemrograman Kotlin dan/atau Java. Branch aktif yang digunakan adalah main, serta remote repository telah terkonfigurasi ke origin dengan URL GitHub yang valid. Struktur proyek menunjukkan penggunaan pendekatan pengembangan Android konvensional, yaitu XML layout yang dikombinasikan dengan Activity, bukan Jetpack Compose. Hal ini mengindikasikan aplikasi dirancang dengan pola standar Android yang masih umum digunakan dan mudah dipahami.

UI dan Tata Letak Aplikasi

Layout utama berada pada file `app/src/main/res/layout/activity_main.xml` dengan `ConstraintLayout` sebagai root layout. Di dalamnya terdapat sebuah `TextView` dengan id `warningTextView` yang menampilkan teks dari resource `@string/proximity_active` dan ukuran teks 20sp. Posisi `TextView` diatur menggunakan constraint ke sisi atas dan sisi start parent, sehingga secara teknis sudah valid karena memenuhi dua sumbu (horizontal dan vertikal). Namun, tata letak masih sangat sederhana karena belum menerapkan margin, padding, maupun styling visual yang konsisten untuk meningkatkan keterbacaan dan estetika antarmuka.

Indikasi Fungsional Aplikasi

Penamaan resource string `proximity_active` serta id `warningTextView` memberikan indikasi kuat bahwa aplikasi berkaitan dengan penggunaan sensor proximity. Atribut `tools:context=".MainActivity"` menegaskan bahwa terdapat kelas `MainActivity` yang

bertanggung jawab mengelola layout ini. Besar kemungkinan MainActivity juga menangani logika pembacaan sensor proximity melalui SensorManager dan mengubah tampilan UI berdasarkan kondisi sensor.

Kualitas Kode dan Manajemen Resource

Penggunaan resource string sudah sesuai dengan praktik yang baik karena mendukung maintainability dan internasionalisasi. Pastikan file strings.xml tersedia dan berisi definisi proximity_active. Penamaan id dan resource sudah cukup deskriptif, namun masih dapat ditingkatkan konsistensinya, misalnya dengan konvensi penamaan yang lebih seragam seperti tvWarningProximity. Selain itu, belum terlihat penggunaan styles.xml atau dimens.xml; sebaiknya ukuran teks, margin, dan warna dipusatkan dalam resource tersebut agar konsisten dan mudah dirawat. Dari sisi aksesibilitas, perlu diperhatikan kontras teks serta penambahan atribut pendukung jika nantinya terdapat elemen non-teks.

Arsitektur dan Pengelolaan Logika Aplikasi

Dengan layout yang sederhana, kemungkinan besar seluruh logika sensor ditempatkan langsung di MainActivity. Untuk skala proyek yang lebih baik dan kemudahan pengujian, logika sensor sebaiknya dipisahkan ke kelas tersendiri atau menggunakan ViewModel yang lifecycle-aware. Pengelolaan sensor proximity harus memperhatikan siklus hidup Activity, khususnya proses register dan unregister listener, untuk mencegah kebocoran memori dan konsumsi baterai berlebih. Sensor proximity memang tidak memerlukan permission khusus, namun pengelolaan lifecycle tetap menjadi aspek penting.

Build, Dependency, dan Pengujian

Karena proyek menggunakan Gradle, konfigurasi pada build.gradle (project dan module) perlu ditinjau untuk memastikan versi SDK, plugin Kotlin, dan dependency sudah sesuai standar terbaru. Disarankan menggunakan ViewBinding atau DataBinding untuk menghindari penggunaan findViewById yang rawan error. Dari sisi pengujian, unit test berbasis JUnit dapat diterapkan untuk logika non-UI, sedangkan Espresso digunakan untuk pengujian UI. Untuk pengujian sensor, diperlukan abstraksi atau mock, misalnya dengan Robolectric atau implementasi SensorManager palsu.

Keamanan, Performa, dan Pengalaman Pengguna

Aplikasi perlu menangani perubahan konfigurasi seperti rotasi layar dengan benar, terutama saat sensor sedang aktif. Pembaruan UI harus selalu dilakukan di main thread untuk menghindari crash. Pengalaman pengguna dapat ditingkatkan dengan menampilkan state yang jelas ketika sensor aktif atau tidak aktif, serta menambahkan animasi ringan yang tidak membebani performa. Terakhir, optimasi ukuran APK melalui R8 atau ProGuard dapat dipertimbangkan agar aplikasi lebih efisien saat didistribusikan.

POST-TEST

1. Setelah Dari praktikum diatas apa fungsi dari potongan kode dibawah ini private fun resetUI() { turnOffFlash() mainLayout.setBackgroundColor(Color.WHITE) textView.text = "Proximity Sensor Active" }

Jawab : Fungsi resetUI() digunakan untuk mengembalikan tampilan aplikasi ke kondisi awal atau kondisi normal (idle). Di dalam fungsi ini, flashlight dimatikan melalui pemanggilan turnOffFlash(), warna latar belakang layout utama (mainLayout) diatur menjadi putih, dan

teks pada textView diubah menjadi “*Proximity Sensor Active*”. Dengan demikian, fungsi ini berperan sebagai mekanisme reset tampilan ketika sensor proximity tidak mendeteksi adanya objek di dekat perangkat.

2. Dari praktikum diatas juga terdapat perpindahan menu dengan menggunakan fragment dan activity, jelaskan perbedaan keduanya?

Jawab : Aplikasi hasil praktikum merespons perubahan jarak objek berdasarkan data dari sensor proximity. Ketika tidak ada objek yang terdeteksi, aplikasi menampilkan kondisi default berupa latar belakang putih, teks “*Proximity Sensor Active*”, dan flashlight dalam keadaan mati. Sebaliknya, ketika sensor mendeteksi adanya objek di dekat perangkat, aplikasi akan mengubah kondisi UI, seperti mengganti teks peringatan, mengubah warna latar belakang layar, dan dapat menyalakan flashlight sebagai indikator bahwa sensor sedang aktif mendeteksi objek.

3. Menurut Anda apakah navigasi dapat di geser dari kanan kekiri? Kalau bisa bagaimana caranya?

Jawab : Bagian kode yang perlu diubah berada pada logika pendeteksian sensor di MainActivity, biasanya di dalam metode `onSensorChanged()` atau listener sensor proximity. Tepatnya pada percabangan (if) yang menangani kondisi saat nilai sensor menunjukkan objek terdeteksi. Pada bagian inilah pengaturan warna latar belakang layar harus diubah menjadi biru, misalnya dengan memanggil `mainLayout.setBackgroundColor(Color.BLUE)`. Sementara itu, kondisi sebaliknya (tidak ada objek) tetap memanggil fungsi `resetUI()` untuk mengembalikan tampilan ke keadaan normal.

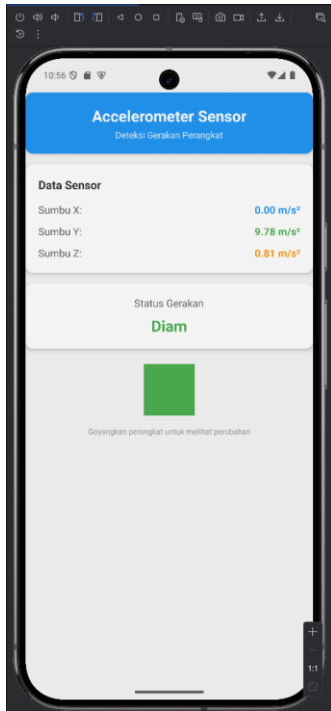
LATIHAN/TUGAS

Buatlah sebuah aplikasi serupa dengan menggunakan sensor yang ada pada smartphone masing-masing (selain proximity)

Source Code :

https://github.com/MuhammadRizalNurfirdaus/Praktikum_BP3/tree/a46c56355976aa7151ea1820660716dedb652110/Tugas_m10

Hasil Run :



Analisis : Aplikasi ini merupakan implementasi pemanfaatan sensor accelerometer pada perangkat Android yang bertujuan untuk mendeteksi serta memvisualisasikan pergerakan perangkat secara real-time. Pengembangan aplikasi dilakukan menggunakan bahasa pemrograman Kotlin dan memanfaatkan Android Sensor API sebagai antarmuka utama untuk mengakses data sensor. Fokus utama aplikasi adalah membaca nilai akselerasi pada tiga sumbu (X, Y, dan Z) serta menerjemahkannya menjadi informasi visual yang mudah dipahami oleh pengguna.

Dari sisi struktur kelas, MainActivity mewarisi AppCompatActivity dan mengimplementasikan interface SensorEventListener. Dengan mengimplementasikan interface ini, kelas wajib menyediakan dua method callback utama, yaitu onSensorChanged() dan onAccuracyChanged(). Implementasi ini memungkinkan aplikasi menerima notifikasi secara langsung ketika terjadi perubahan nilai sensor maupun perubahan tingkat akurasi sensor, sehingga aplikasi dapat merespons data sensor secara real-time.

Pada tahap inisialisasi, aplikasi mendeklarasikan beberapa komponen penting. SensorManager digunakan sebagai pengelola utama sensor perangkat, sementara objek Sensor dengan tipe TYPE_ACCELEROMETER merepresentasikan sensor akselerometer yang digunakan. Untuk antarmuka pengguna, aplikasi memanfaatkan beberapa TextView yang berfungsi menampilkan nilai akselerasi pada sumbu X, Y, dan Z, serta satu TextView tambahan untuk menampilkan status pergerakan. Selain itu, terdapat sebuah View yang digunakan sebagai indikator visual berupa

perubahan warna latar. Variabel lastX, lastY, dan lastZ digunakan untuk menyimpan nilai sensor sebelumnya sebagai dasar perhitungan perubahan akselerasi, sedangkan konstanta SHAKE_THRESHOLD dengan nilai 15.0f berfungsi sebagai ambang batas untuk mendeteksi guncangan kuat.

Pada lifecycle method onCreate(), aplikasi melakukan serangkaian inisialisasi awal. Mode tampilan edge-to-edge diaktifkan agar aplikasi dapat memanfaatkan seluruh area layar secara modern. Selanjutnya, window insets diatur agar konten aplikasi tidak tertutup oleh system bars seperti status bar dan navigation bar, dengan menerapkan padding secara dinamis. Setelah itu, seluruh komponen UI dihubungkan dengan layout XML menggunakan findViewById(). Inisialisasi SensorManager dilakukan melalui getSystemService(), dan sensor accelerometer diambil menggunakan getDefaultSensor(Sensor.TYPE_ACCELEROMETER). Aplikasi juga melakukan pengecekan ketersediaan sensor dan memberikan umpan balik kepada pengguna melalui pesan Toast, baik berupa notifikasi sensor siap digunakan maupun peringatan jika sensor tidak tersedia.

Pengelolaan lifecycle sensor dilakukan secara tepat melalui method onResume() dan onPause(). Pada onResume(), aplikasi mendaftarkan listener sensor menggunakan registerListener() dengan delay SENSOR_DELAY_NORMAL, yang memberikan keseimbangan antara responsivitas aplikasi dan efisiensi konsumsi baterai. Sebaliknya, pada onPause(), listener sensor dilepas menggunakan unregisterListener() untuk mencegah pembacaan sensor yang tidak perlu ketika aplikasi tidak berada di foreground, sehingga lebih hemat daya dan aman dari potensi memory leak.

Logika utama aplikasi terletak pada method onSensorChanged(). Method ini terlebih dahulu memastikan bahwa event yang diterima berasal dari sensor accelerometer. Nilai akselerasi pada sumbu X, Y, dan Z diambil dari array event.values dan kemudian diformat menjadi dua angka desimal sebelum ditampilkan ke TextView dengan satuan m/s². Selanjutnya, aplikasi menghitung perubahan akselerasi pada masing-masing sumbu dengan membandingkan nilai saat ini dengan nilai sebelumnya menggunakan fungsi absolut. Berdasarkan nilai perubahan tersebut, aplikasi mengklasifikasikan status pergerakan ke dalam tiga kategori, yaitu “Bergerak Kuat!” jika melebihi ambang batas SHAKE_THRESHOLD, “Bergerak” untuk pergerakan sedang, dan “Diam” jika tidak terdapat pergerakan signifikan. Setiap kategori ditampilkan dengan warna yang berbeda untuk memperkuat informasi visual.

Setelah klasifikasi dilakukan, nilai akselerasi saat ini disimpan kembali ke variabel lastX, lastY, dan lastZ sebagai referensi untuk pembacaan sensor berikutnya. Aplikasi juga menghitung intensitas pergerakan rata-rata, yang kemudian digunakan untuk memperbarui warna latar indikator visual, sehingga pengguna dapat melihat representasi pergerakan secara lebih intuitif.

Selain logika utama, aplikasi memiliki beberapa method pendukung. Method updateStatus() bertugas memperbarui teks dan warna pada TextView status dengan memanfaatkan Color.parseColor() untuk mengonversi kode warna heksadesimal. Method updateIndicatorColor() memiliki fungsi serupa, namun difokuskan pada perubahan warna latar View indikator berdasarkan intensitas pergerakan. Sementara itu, method onAccuracyChanged() telah diimplementasikan sesuai kontrak interface, meskipun saat ini belum digunakan secara aktif. Method ini tetap penting sebagai fondasi pengembangan lanjutan, misalnya untuk memberikan informasi kepada pengguna terkait perubahan akurasi sensor.

Secara keseluruhan, aplikasi ini menunjukkan implementasi sensor accelerometer yang terstruktur dan sesuai dengan praktik terbaik pengembangan Android. Pengelolaan lifecycle sensor dilakukan dengan benar, antarmuka pengguna memberikan visualisasi data yang jelas, dan

klasifikasi pergerakan disajikan secara informatif. Dengan struktur kode yang rapi dan logika yang mudah dikembangkan, aplikasi ini dapat menjadi dasar yang kuat untuk pengembangan fitur sensor yang lebih kompleks di masa mendatang.